ACADEMIC PROJECT REPORT

LabTrack: Lab Equipment & Component Borrowing System

Full Stack Laboratory Hardware & Inventory Tracking System

1. Title of the Project & Team Members

LabTrack: Lab Equipment & Component Borrowing System

Sourabh Gupta
PRN: 25070122191

Sneha Sah
PRN: 25070122189

Shreeya Alwar
PRN: 25070122180

Shreyash Deshmukh
PRN: 26070122511

2. Motivation for Selection of the Topic

In our college labs, borrowing hardware like Arduino boards, Raspberry Pis, multimeters, or basic sensor modules has always been a messy manual process. Students have to hunt down physical paper slips, get manual signatures, and wait for the lab assistant to note down serial numbers inside a thick physical register book. Half the time, nobody knows who took which component, whether an item is broken, or when someone is supposed to return it.

We experienced this headache firsthand while working on our own lab experiments and semester projects. Boards would go missing right before submission week, and lab staff had no easy way to track down overdue items without personally texting or calling students.

We chose this project to fix that exact daily bottleneck. Instead of dealing with lost paperwork and untracked hardware cupboards, we wanted to build a straightforward web platform where any student can check stock live from their phone or laptop, upload an ID verification slip, and check out items without chaos. For the lab in-charge, it turns endless notebook entries into a clean screen where stock, returns, and student history are organized in one place.

3. Objectives of the Project

Our main goals while building LabTrack were:

- **Eliminate Paper Register Books:** Create a simple web interface so students can browse hardware inventory, see real-time available quantities, and submit requisition requests online instead of filling paper chits.
- **Cut Down Hardware Losses via Automated Emails:** Set up automated email receipts through Node.js so students get an instant confirmation with return deadlines, plus an admin-triggered overdue reminder when an item is past its due date.
- **Keep an Unalterable Server Audit Trail:** Use Node's built-in file system (`fs`) and event emitters to write every single checkout, issue, and return action into an append-only log file (`audit_trail.log`), making sure no records can be quietly deleted or tampered with.
- **Give Admins Real-Time Stock & Department Stats:** Build a backend aggregation pipeline in MongoDB to calculate department-wise equipment usage and category stock totals, displaying live stats directly on the admin dashboard.
- **Protect the App with Strict Validation & Auth:** Restrict actions using JWT tokens and role checks (Student vs. Admin), sanitize user input with Regular Expressions to block malicious `<script>` tags, and enforce an exact 11-digit PRN format for student accounts.

4. Customer Base / Target Users

The system is built specifically for college engineering laboratories, maker spaces, and department research workshops. Our target audience falls into two main groups:

> **1. College Students (End Users)**
> Undergraduate and postgraduate students from departments like ECE, CSE, Robotics, and Mechanical who need development boards and instruments for lab practicals, semester capstones, or hackathons. They use LabTrack to check if an item is sitting on the lab shelf before walking over, file borrow requests with their 11-digit PRN, and monitor when their gear must be returned.

> **2. Lab Assistants & Faculty Administrators (Staff Users)**
> The laboratory technicians and professors managing inventory in rooms like Lab 301, 302, and 305. They use the platform to verify student requests, check uploaded ID slips, mark items as returned to restore inventory stock instantly, email students holding overdue equipment, and inspect branch-wise usage numbers.

5. Summary of the Project

LabTrack is a full-stack web application designed from the ground up using Node.js, Express, MongoDB, and Vanilla JavaScript. We purposefully avoided bloated front-end frameworks so the entire application remains lightweight, fast-loading, and easy to deploy on a single web service.

Frontend Architecture: The user interface is written in semantic HTML5 (`<header>`, `<nav>`, `<section>`, `<article>`, `<footer>`). We used CSS Grid for the hardware catalog cards so they automatically adjust across mobile, tablet, and widescreen monitors, while Flexbox handles the top stats bar and navigation tabs. Client-side JavaScript manages modal dialogs, search queries, and dynamic DOM updates. Before sending any request to the server, regular expressions strip out script tags and verify that the PRN strictly follows the 11-digit format (`^[0-9]{11}$`).

Backend & Node.js Core: The server runs on Express.js with custom middleware. To meet core Node.js requirements, we used the native `EventEmitter` and non-blocking `fs.appendFile` in `src/events/auditLogger.js`. Every time a component is issued or returned, an event fires in the background and writes a timestamped record to `logs/audit_trail.log` without stalling HTTP response times. An administrative endpoint (`GET /api/borrow/logs`) reads this file back using `fs.readFile` to populate the audit viewer modal. File uploads for student IDs and requisition documents are handled using `multer`, storing uploads safely in an `/uploads` folder with unique timestamped filenames.

Database Design & Aggregations: We designed three interconnected collections in MongoDB using Mongoose: `Users`, `Components`, and `BorrowRecords`. Instead of running basic database lookups, our analytics route runs a multi-stage MongoDB Aggregation Pipeline utilizing `$group`, `$sum`, `$project`, and `$lookup`. This lets the server calculate stock utilization rates and sum up total borrowings by academic branch (CSE, ECE, Robotics, etc.) in a single database operation.

Security, Emails & Deployment: Authentication relies on JSON Web Tokens (JWT) stored securely on the client, with passwords salted and hashed using `bcryptjs`. For automated notifications, the system connects via `nodemailer` to send issue confirmations and overdue notices (with automatic Ethereal test fallback for local evaluation). All port numbers, database URIs, and secrets are loaded through `dotenv`. Running `npm start` serves both the static frontend and REST API routes together, allowing simple hosting on platforms like Render.

6. Git Repository Link

```
https://github.com/sourabhguptabtech2025/LabTrack-Equipment-Borrowing-System
```